

CoreGuard Gateway Admin Auth - Complete Code Guide

Phase A1/A2 implementation reference with migration, seed command, repository, JWT/refresh-token rotation, rate limiting, middleware, router wiring, cleanup, and tests.

Project	CoreGuard Gateway
Preset	compact_reference_guide
Scope	Admin auth implementation reference

1. Overview

This guide is tailored to the current CoreGuard Gateway repo shape: Go module edgecore, Gin routers, database/sql with lib/pq, zerolog logging, migrations under migrations, and existing packages under internal/auth, internal/storage, and internal/admin. The code below is intended as implementation-ready reference. Small naming adjustments may be needed if you choose different constructor wiring.

2. Dependencies

Add these direct dependencies if they are not already direct in go.mod. The JWT package already exists indirectly, but bcrypt should be direct once used by application code.

powershell

```
go get golang.org/x/crypto/bcrypt
# jwt/v5 already exists in this repo indirectly; direct usage is okay.
# This guide avoids a Go UUID package by using Postgres gen_random_uuid().
```

3. migrations/0004_admin_auth.up.sql

sql

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE IF NOT EXISTS admin_users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    username TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    failed_login_attempts INT NOT NULL DEFAULT 0,
    locked_until TIMESTAMPTZ NULL,
    last_login_at TIMESTAMPTZ NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS refresh_tokens (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    admin_user_id UUID NOT NULL REFERENCES admin_users(id) ON DELETE CASCADE,
    token_hash TEXT UNIQUE NOT NULL,
    expires_at TIMESTAMPTZ NOT NULL,
    revoked_at TIMESTAMPTZ NULL,
    ip_address TEXT NULL,
    user_agent TEXT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS idx_admin_users_username
    ON admin_users(username);

CREATE INDEX IF NOT EXISTS idx_refresh_tokens_token_hash
    ON refresh_tokens(token_hash);

CREATE INDEX IF NOT EXISTS idx_refresh_tokens_admin_user_id
    ON refresh_tokens(admin_user_id);
```

```
CREATE INDEX IF NOT EXISTS idx_refresh_tokens_expires_at
ON refresh_tokens(expires_at);
```

4. migrations/0004_admin_auth.down.sql

sql

```
DROP TABLE IF EXISTS refresh_tokens;
DROP TABLE IF EXISTS admin_users;
-- Keep pgcrypto installed because other migrations/features may depend on it.
```

5. internal/auth/errors.go

go

```
package auth

import "errors"

var (
    ErrInvalidCredentials = errors.New("invalid credentials")
    ErrUserLocked         = errors.New("admin user locked")
    ErrTokenMissing       = errors.New("token missing")
    ErrTokenExpired       = errors.New("token expired")
    ErrTokenRevoked       = errors.New("token revoked")
    ErrInvalidToken       = errors.New("invalid token")
    ErrRateLimited        = errors.New("rate limited")
)
```

6. internal/auth/tokens.go

go

```
package auth

import (
    "crypto/rand"
    "crypto/sha256"
    "encoding/base64"
    "encoding/hex"
    "errors"
    "fmt"
    "time"

    "github.com/golang-jwt/jwt/v5"
)

const (
    AdminRole           = "admin"
    AccessTokenTTL      = 15 * time.Minute
    RefreshTokenTTL     = 7 * 24 * time.Hour
    MinJWTSecretByteLength = 32
)

type AdminClaims struct {
    Username string `json:"username"`
    Role     string `json:"role"`
    jwt.RegisteredClaims
}

type AdminTokenManager struct {
    secret []byte
    issuer string
}

func NewAdminTokenManager(secret, issuer string) (*AdminTokenManager, error) {
    if len([]byte(secret)) < MinJWTSecretByteLength {
        return nil, fmt.Errorf("JWT_SECRET must be at least %d bytes", MinJWTSecretByteLength)
    }
    if issuer == "" {
        issuer = "coreguard-admin"
    }
    return &AdminTokenManager{secret: []byte(secret), issuer: issuer}, nil
}

func (m *AdminTokenManager) CreateAdminAccessToken(userID, username string) (string, error) {
    now := time.Now().UTC()
    claims := AdminClaims{
```

```

        Username: username,
        Role:      AdminRole,
        RegisteredClaims: jwt.RegisteredClaims{
            Subject:  userID,
            Issuer:    m.issuer,
            IssuedAt:  jwt.NewNumericDate(now),
            ExpiresAt: jwt.NewNumericDate(now.Add(AccessTokenTTL)),
        },
    }
    return jwt.NewWithClaims(jwt.SigningMethodHS256, claims).SignedString(m.secret)
}

func (m *AdminTokenManager) ValidateAdminAccessToken(raw string) (*AdminClaims, error) {
    token, err := jwt.ParseWithClaims(raw, &AdminClaims{}, func(t *jwt.Token) (interface{}, error) {
        if t.Method != jwt.SigningMethodHS256 {
            return nil, fmt.Errorf("unexpected signing method: %v", t.Header["alg"])
        }
        return m.secret, nil
    })
    if err != nil {
        if errors.Is(err, jwt.ErrTokenExpired) {
            return nil, ErrTokenExpired
        }
        return nil, ErrInvalidToken
    }
    claims, ok := token.Claims.(*AdminClaims)
    if !ok || !token.Valid || claims.Role != AdminRole || claims.Subject == "" {
        return nil, ErrInvalidToken
    }
    return claims, nil
}

func GenerateRefreshToken() (string, error) {
    b := make([]byte, 32)
    if _, err := rand.Read(b); err != nil {
        return "", err
    }
    return base64.RawURLEncoding.EncodeToString(b), nil
}

func HashToken(raw string) string {
    sum := sha256.Sum256([]byte(raw))
    return hex.EncodeToString(sum[:])
}

```

7. internal/storage/admin_auth_repo.go

```

go
package storage

import (
    "context"
    "database/sql"
    "errors"
    "time"
)

type AdminUser struct {
    ID            string
    Username      string
    PasswordHash  string
    FailedLoginAttempts int
    LockedUntil   sql.NullTime
    LastLoginAt   sql.NullTime
    CreatedAt     time.Time
}

type RefreshToken struct {
    ID            string
    AdminUserID   string
    TokenHash     string
    ExpiresAt     time.Time
    RevokedAt     sql.NullTime
    IPAddress     sql.NullString
    UserAgent     sql.NullString
    CreatedAt     time.Time
}

type AdminAuthRepo struct { db *sql.DB }

func NewAdminAuthRepo(db *sql.DB) *AdminAuthRepo { return &AdminAuthRepo{db: db} }

```

```

func (r *AdminAuthRepo) FindAdminUserByUsername(ctx context.Context, username string) (*AdminUser, error) {
    const q = `SELECT id, username, password_hash, failed_login_attempts, locked_until,
        last_login_at, created_at FROM admin_users WHERE username=$1`
    var u AdminUser
    err := r.db.QueryRowContext(ctx, q, username).Scan(&u.ID, &u.Username, &u.PasswordHash,
        &u.FailedLoginAttempts, &u.LockedUntil, &u.LastLoginAt, &u.CreatedAt)
    if errors.Is(err, sql.ErrNoRows) { return nil, sql.ErrNoRows }
    if err != nil { return nil, err }
    return &u, nil
}

func (r *AdminAuthRepo) CreateRefreshToken(ctx context.Context, userID, tokenHash string, expiresAt time.Time,
    ip, ua string) error {
    const q = `INSERT INTO refresh_tokens (admin_user_id, token_hash, expires_at, ip_address, user_agent)
        VALUES ($1,$2,$3,$4,$5)`
    _, err := r.db.ExecContext(ctx, q, userID, tokenHash, expiresAt, nullText(ip), nullText(ua))
    return err
}

func (r *AdminAuthRepo) FindRefreshToken(ctx context.Context, tokenHash string) (*RefreshToken, error) {
    const q = `SELECT id, admin_user_id, token_hash, expires_at, revoked_at, ip_address, user_agent, created_at
        FROM refresh_tokens WHERE token_hash=$1`
    var t RefreshToken
    err := r.db.QueryRowContext(ctx, q, tokenHash).Scan(&t.ID, &t.AdminUserID, &t.TokenHash,
        &t.ExpiresAt, &t.RevokedAt, &t.IPAddress, &t.UserAgent, &t.CreatedAt)
    if errors.Is(err, sql.ErrNoRows) { return nil, sql.ErrNoRows }
    if err != nil { return nil, err }
    return &t, nil
}

func (r *AdminAuthRepo) RevokeRefreshToken(ctx context.Context, tokenHash string) error {
    _, err := r.db.ExecContext(ctx, `UPDATE refresh_tokens SET revoked_at=now()
        WHERE token_hash=$1 AND revoked_at IS NULL`, tokenHash)
    return err
}

func (r *AdminAuthRepo) RevokeAllRefreshTokensForUser(ctx context.Context, userID string) error {
    _, err := r.db.ExecContext(ctx, `UPDATE refresh_tokens SET revoked_at=now()
        WHERE admin_user_id=$1 AND revoked_at IS NULL`, userID)
    return err
}

func (r *AdminAuthRepo) RotateRefreshToken(ctx context.Context, oldHash, newHash, userID string, newExpiresAt
    time.Time, ip, ua string) error {
    tx, err := r.db.BeginTx(ctx, nil)
    if err != nil { return err }
    defer tx.Rollback()

    res, err := tx.ExecContext(ctx, `UPDATE refresh_tokens SET revoked_at=now()
        WHERE token_hash=$1 AND revoked_at IS NULL`, oldHash)
    if err != nil { return err }
    affected, err := res.RowsAffected()
    if err != nil { return err }
    if affected != 1 { return sql.ErrNoRows }

    _, err = tx.ExecContext(ctx, `INSERT INTO refresh_tokens
        (admin_user_id, token_hash, expires_at, ip_address, user_agent)
        VALUES ($1,$2,$3,$4,$5)`, userID, newHash, newExpiresAt, nullText(ip), nullText(ua))
    if err != nil { return err }
    return tx.Commit()
}

func (r *AdminAuthRepo) PruneExpiredTokens(ctx context.Context) error {
    _, err := r.db.ExecContext(ctx, `DELETE FROM refresh_tokens
        WHERE expires_at < now() OR revoked_at < now() - interval '7 days'`)
    return err
}

func (r *AdminAuthRepo) UpdateAdminLoginSuccess(ctx context.Context, userID string) error {
    _, err := r.db.ExecContext(ctx, `UPDATE admin_users SET failed_login_attempts=0,
        locked_until=NULL, last_login_at=now() WHERE id=$1`, userID)
    return err
}

func (r *AdminAuthRepo) IncrementAdminLoginFailure(ctx context.Context, username string) error {
    _, err := r.db.ExecContext(ctx, `UPDATE admin_users
        SET failed_login_attempts=failed_login_attempts+1 WHERE username=$1`, username)
    return err
}

func (r *AdminAuthRepo) ResetAdminLoginFailures(ctx context.Context, userID string) error {
    _, err := r.db.ExecContext(ctx, `UPDATE admin_users

```

```

        SET failed_login_attempts=0, locked_until=NULL WHERE id=$1`, userID)
    return err
}

func (r *AdminAuthRepo) LockAdminUser(ctx context.Context, username string, until time.Time) error {
    _, err := r.db.ExecContext(ctx, `UPDATE admin_users SET locked_until=$2 WHERE username=$1`, username, until)
    return err
}

func nullText(v string) sql.NullString { return sql.NullString{String: v, Valid: v != ""} }

```

8. cmd/adminseed/main.go

go

```

package main

import (
    "context"
    "database/sql"
    "errors"
    "fmt"
    "log"
    "os"
    "strings"

    _ "github.com/lib/pq"
    "golang.org/x/crypto/bcrypt"
)

func main() {
    username := strings.TrimSpace(os.Getenv("ADMIN_USERNAME"))
    password := os.Getenv("ADMIN_PASSWORD")
    dsn := os.Getenv("POSTGRES_DSN")
    if username == "" || password == "" || dsn == "" {
        log.Fatal("ADMIN_USERNAME, ADMIN_PASSWORD, and POSTGRES_DSN are required")
    }
    if password == "admin123" {
        log.Println("warning: admin123 is development-only; do not use it in production")
    } else if len(password) < 12 {
        log.Fatal("ADMIN_PASSWORD must be at least 12 characters outside local development")
    }

    db, err := sql.Open("postgres", dsn)
    if err != nil { log.Fatal(err) }
    defer db.Close()

    ctx := context.Background()
    hash, err := bcrypt.GenerateFromPassword([]byte(password), bcrypt.DefaultCost)
    if err != nil { log.Fatal(err) }

    const q = `INSERT INTO admin_users (username, password_hash)
        VALUES ($1,$2) ON CONFLICT (username) DO NOTHING RETURNING id`
    var id string
    err = db.QueryRowContext(ctx, q, username, string(hash)).Scan(&id)
    if errors.Is(err, sql.ErrNoRows) {
        fmt.Printf("admin user %q already exists\n", username)
        return
    }
    if err != nil { log.Fatal(err) }
    fmt.Printf("admin user %q created with id %s\n", username, id)
}

```

9. internal/admin/middleware/rate_limit.go

go

```

package middleware

import (
    "net"
    "strings"
    "sync"
    "time"

    "github.com/gin-gonic/gin"
)

```

```

type LoginRateLimiter struct {
    mu      sync.Mutex
    failures map[string][]time.Time
    limit    int
    window   time.Duration
}

func NewLoginRateLimiter(limit int, window time.Duration) *LoginRateLimiter {
    return &LoginRateLimiter{failures: map[string][]time.Time{}, limit: limit, window: window}
}

func (l *LoginRateLimiter) TooManyFailures(ip string) bool {
    l.mu.Lock(); defer l.mu.Unlock()
    now := time.Now()
    kept := l.compact(ip, now)
    return len(kept) >= l.limit
}

func (l *LoginRateLimiter) RecordFailure(ip string) {
    l.mu.Lock(); defer l.mu.Unlock()
    now := time.Now()
    kept := l.compact(ip, now)
    l.failures[ip] = append(kept, now)
}

func (l *LoginRateLimiter) Reset(ip string) {
    l.mu.Lock(); defer l.mu.Unlock()
    delete(l.failures, ip)
}

func (l *LoginRateLimiter) compact(ip string, now time.Time) []time.Time {
    cutoff := now.Add(-l.window)
    attempts := l.failures[ip]
    kept := attempts[:0]
    for _, t := range attempts {
        if t.After(cutoff) { kept = append(kept, t) }
    }
    l.failures[ip] = kept
    return kept
}

func ClientIP(c *gin.Context) string {
    host, _, err := net.SplitHostPort(c.Request.RemoteAddr)
    if err == nil && host != "" { return host }
    return strings.TrimSpace(c.Request.RemoteAddr)
}

// Only use X-Forwarded-For here if your trusted reverse proxy behavior is configured.
// Until then, RemoteAddr is safer than trusting spoofable headers.

```

10. internal/admin/handler/auth_handler.go

ao

```

package handler

import (
    "database/sql"
    "errors"
    "net/http"
    "time"

    adminmw "edgecore/internal/admin/middleware"
    authpkg "edgecore/internal/auth"
    "edgecore/internal/storage"

    "github.com/gin-gonic/gin"
    "github.com/rs/zerolog"
    "golang.org/x/crypto/bcrypt"
)

const (
    maxLoginFailures = 5
    lockoutDuration  = 15 * time.Minute
    maxAuthBodyBytes = 1 << 20
)

type AuthHandler struct {
    repo      *storage.AdminAuthRepo
    tokens    *authpkg.AdminTokenManager
    limiter   *adminmw.LoginRateLimiter
    logger    zerolog.Logger
}

```

```

}

func NewAuthHandler(repo *storage.AdminAuthRepo, tokens *authpkg.AdminTokenManager, limiter
*adminmw.LoginRateLimiter, logger zerolog.Logger) *AuthHandler {
    return &AuthHandler{repo: repo, tokens: tokens, limiter: limiter, logger: logger}
}

type loginRequest struct { Username string `json:"username" binding:"required"`; Password string
`json:"password" binding:"required"` }
type refreshRequest struct { RefreshToken string `json:"refresh_token" binding:"required"` }
type tokenResponse struct { AccessToken string `json:"access_token"`; RefreshToken string
`json:"refresh_token"`; ExpiresIn int `json:"expires_in"`; TokenType string `json:"token_type"` }

func (h *AuthHandler) Login(c *gin.Context) {
    c.Request.Body = http.MaxBytesReader(c.Writer, c.Request.Body, maxAuthBodyBytes)
    ip := adminmw.ClientIP(c)
    if h.limiter.TooManyFailures(ip) {
        h.logger.Warn().Str("ip", ip).Msg("admin login rate limited")
        c.JSON(http.StatusTooManyRequests, gin.H{"error": "too many login attempts"})
        return
    }

    var req loginRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": "invalid request"})
        return
    }

    user, err := h.repo.FindAdminUserByUsername(c.Request.Context(), req.Username)
    if errors.Is(err, sql.ErrNoRows) {
        h.limiter.RecordFailure(ip)
        c.JSON(http.StatusUnauthorized, gin.H{"error": "invalid username or password"})
        return
    }
    if err != nil { h.internal(c, err); return }

    if user.LockedUntil.Valid && user.LockedUntil.Time.After(time.Now()) {
        c.JSON(http.StatusLocked, gin.H{"error": "admin account locked"})
        return
    }

    if bcrypt.CompareHashAndPassword([]byte(user.PasswordHash), []byte(req.Password)) != nil {
        _ = h.repo.IncrementAdminLoginFailure(c.Request.Context(), req.Username)
        h.limiter.RecordFailure(ip)
        if user.FailedLoginAttempts+1 >= maxLoginFailures {
            until := time.Now().Add(lockoutDuration)
            _ = h.repo.LockAdminUser(c.Request.Context(), req.Username, until)
            h.logger.Warn().Str("username", req.Username).Str("ip", ip).Time("until", until).Msg("admin locked")
        }
        c.JSON(http.StatusUnauthorized, gin.H{"error": "invalid username or password"})
        return
    }

    h.limiter.Reset(ip)
    if err := h.repo.UpdateAdminLoginSuccess(c.Request.Context(), user.ID); err != nil { h.internal(c, err);
        return }
    h.issueTokens(c, user.ID, user.Username)
}

func (h *AuthHandler) Refresh(c *gin.Context) {
    c.Request.Body = http.MaxBytesReader(c.Writer, c.Request.Body, maxAuthBodyBytes)
    var req refreshRequest
    if err := c.ShouldBindJSON(&req); err != nil { c.JSON(http.StatusBadRequest, gin.H{"error": "invalid
        request"}); return }

    oldHash := authpkg.HashToken(req.RefreshToken)
    token, err := h.repo.FindRefreshToken(c.Request.Context(), oldHash)
    if errors.Is(err, sql.ErrNoRows) { c.JSON(http.StatusUnauthorized, gin.H{"error": "invalid refresh token"});
        return }
    if err != nil { h.internal(c, err); return }
    if token.RevokedAt.Valid { h.logger.Warn().Str("admin_user_id", token.AdminUserID).Msg("revoked refresh
        token reused"); c.JSON(http.StatusUnauthorized, gin.H{"error": "invalid refresh token"}); return }
    if time.Now().After(token.ExpiresAt) { c.JSON(http.StatusUnauthorized, gin.H{"error": "refresh token
        expired"}); return }

    newRaw, err := authpkg.GenerateRefreshToken(); if err != nil { h.internal(c, err); return }
    newHash := authpkg.HashToken(newRaw)
    newExp := time.Now().Add(authpkg.RefreshTokenTTL)
    ip, ua := adminmw.ClientIP(c), c.Request.UserAgent()
    if err := h.repo.RotateRefreshToken(c.Request.Context(), oldHash, newHash, token.AdminUserID, newExp, ip,
        ua); err != nil { h.internal(c, err); return }

    access, err := h.tokens.CreateAdminAccessToken(token.AdminUserID, "")

```

```

    if err != nil { h.internal(c, err); return }
    c.JSON(http.StatusOK, tokenResponse{AccessToken: access, RefreshToken: newRaw, ExpiresIn:
        int(authpkg.AccessTokenTTL.Seconds()), TokenType: "Bearer"})
}

func (h *AuthHandler) Logout(c *gin.Context) {
    c.Request.Body = http.MaxBytesReader(c.Writer, c.Request.Body, maxAuthBodyBytes)
    var req refreshRequest
    if err := c.ShouldBindJSON(&req); err != nil { c.JSON(http.StatusBadRequest, gin.H{"error": "invalid
        request"}); return }
    _ = h.repo.RevokeRefreshToken(c.Request.Context(), authpkg.HashToken(req.RefreshToken))
    c.JSON(http.StatusOK, gin.H{"status": "ok"})
}

func (h *AuthHandler) issueTokens(c *gin.Context, userID, username string) {
    access, err := h.tokens.CreateAdminAccessToken(userID, username); if err != nil { h.internal(c, err); return
    }
    refresh, err := authpkg.GenerateRefreshToken(); if err != nil { h.internal(c, err); return }
    exp := time.Now().Add(authpkg.RefreshTokenTTL)
    if err := h.repo.CreateRefreshToken(c.Request.Context(), userID, authpkg.HashToken(refresh), exp,
        adminmw.ClientIP(c), c.Request.UserAgent()); err != nil { h.internal(c, err); return }
    h.logger.Info().Str("admin_user_id", userID).Msg("admin login success")
    c.JSON(http.StatusOK, tokenResponse{AccessToken: access, RefreshToken: refresh, ExpiresIn:
        int(authpkg.AccessTokenTTL.Seconds()), TokenType: "Bearer"})
}

func (h *AuthHandler) internal(c *gin.Context, err error) {
    h.logger.Error().Err(err).Msg("admin auth internal error")
    c.JSON(http.StatusInternalServerError, gin.H{"error": "internal server error"})
}

```

11. Important handler note

For refresh responses, the sample above passes an empty username when creating the new access token because `refresh_tokens` does not store username. Best production-friendly fix: either join `admin_users` in `FindRefreshToken` or add `FindAdminUserByID(ctx, id)` and load username before signing the refreshed access token. The JWT sub and role are the critical authorization checks; username is convenience metadata.

12. internal/admin/middleware/admin_auth.go

```

go
package middleware

import (
    "net/http"
    "strings"

    authpkg "edgecore/internal/auth"
    "github.com/gin-gonic/gin"
)

const AdminUserIDKey = "admin_user_id"
const AdminUsernameKey = "admin_username"

func AdminAuth(tokens *authpkg.AdminTokenManager) gin.HandlerFunc {
    return func(c *gin.Context) {
        header := c.GetHeader("Authorization")
        if header == "" || !strings.HasPrefix(header, "Bearer ") {
            c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error": "missing bearer token"})
            return
        }
        raw := strings.TrimSpace(strings.TrimPrefix(header, "Bearer "))
        claims, err := tokens.ValidateAdminAccessToken(raw)
        if err != nil {
            c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error": "invalid token"})
            return
        }
        if claims.Role != authpkg.AdminRole {
            c.AbortWithStatusJSON(http.StatusForbidden, gin.H{"error": "forbidden"})
            return
        }
        c.Set(AdminUserIDKey, claims.Subject)
        c.Set(AdminUsernameKey, claims.Username)
        c.Next()
    }
}

```


13. internal/admin/router.go wiring pattern

go

```
func NewRouter(logger zerolog.Logger, db *sql.DB, jwtSecret string) http.Handler {
    gin.SetMode(gin.ReleaseMode)
    r := gin.New()
    r.Use(gin.Logger(), gin.Recovery())

    r.GET("/health", healthHandler)
    r.GET("/ready", readyHandler)

    adminTokens, err := auth.NewAdminTokenManager(jwtSecret, "coreguard-admin")
    if err != nil { panic(err) }
    authRepo := storage.NewAdminAuthRepo(db)
    loginLimiter := middleware.NewLoginRateLimiter(5, time.Minute)
    authHandler := handler.NewAuthHandler(authRepo, adminTokens, loginLimiter, logger)

    r.POST("/admin/login", authHandler.Login)
    r.POST("/admin/refresh", authHandler.Refresh)
    r.POST("/admin/logout", authHandler.Logout)

    v1 := r.Group("/admin/v1")
    v1.Use(middleware.AdminAuth(adminTokens))
    {
        routes := v1.Group("/routes")
        routes.GET("", listRoutesHandler)
        routes.POST("", createRouteHandler)
        routes.PUT("/:id", updateRouteHandler)
        routes.DELETE("/:id", deleteRouteHandler)

        // Add protected groups when implemented:
        // v1.Group("/upstreams")
        // v1.Group("/api-keys")
    }

    logger.Debug().Msg("admin router configured")
    return r
}
```

14. internal/admin/server/server.go wiring reminder

Your current NewRouter only accepts logger. To use DB-backed auth, pass db and JWT_SECRET/config into the admin router from the admin app/server construction layer. If the current admin app does not already own db, create it with storage.NewPostgres(logger) before constructing the router.

15. internal/admin/prune.go

go

```
package admin

import (
    "context"
    "time"

    "edgecore/internal/storage"
    "github.com/rs/zerolog"
)

func StartRefreshTokenPruner(ctx context.Context, repo *storage.AdminAuthRepo, logger zerolog.Logger) {
    ticker := time.NewTicker(6 * time.Hour)
    go func() {
        defer ticker.Stop()
        for {
            select {
            case <-ctx.Done():
                return
            case <-ticker.C:
                if err := repo.PruneExpiredTokens(ctx); err != nil {
                    logger.Error().Err(err).Msg("failed to prune refresh tokens")
                }
            }
        }
    }()
}
```

16. Testing checklist

powershell

```
go test ./...

docker compose -f deploy\docker-compose.yml up --build

$env:ADMIN_USERNAME="admin"
$env:ADMIN_PASSWORD="admin123"
$env:POSTGRES_DSN="postgres://..."
go run .\cmd\adminseed

POST http://localhost:9090/admin/login
GET http://localhost:9090/admin/v1/routes Authorization: Bearer <access_token>
POST http://localhost:9090/admin/refresh
POST http://localhost:9090/admin/logout
GET http://localhost:8080/api/admin/v1/routes Authorization: Bearer <access_token>
```

17. Security acceptance checks

text

```
[ ] Passwords are bcrypt-hashed; raw passwords are never logged.
[ ] Refresh tokens are generated with crypto/rand.
[ ] Only SHA-256 refresh token hashes are stored.
[ ] Refresh token rotation revokes old token and inserts new token in one DB transaction.
[ ] Old refresh token fails after refresh.
[ ] Reuse of revoked refresh token is logged as suspicious.
[ ] /admin/login has HTTP-layer rate limiting before bcrypt.
[ ] Account lockout works after repeated failures.
[ ] JWT_SECRET is required, 32+ bytes, and never hardcoded.
[ ] /admin/v1/* requires Bearer JWT with role=admin.
[ ] /health and /ready remain public.
[ ] Logs never include access tokens, refresh tokens, token hashes, or password hashes.
[ ] Expired/revoked refresh token cleanup is scheduled.
```

Final Implementation Notes

Refresh token rotation is included and should be atomic: revoke the old token and insert the new token in one DB transaction.

HTTP-layer login rate limiting is included separately from DB account logout.

JWT key rotation is intentionally deferred; use a required, strong 32+ byte JWT_SECRET and 15-minute access tokens for this phase.

If browser cookie auth is added later, add httpOnly Secure SameSite cookies and CSRF protection then. The current API/Postman design returns JSON tokens.